

GREEN COMPUTING FINAL PROJECT:

ANALYSIS OF 5 MEDICAL DATASET AND APPLICATION OF GREEN STRATEGIES TO REDUCE ENERGY, CO₂ EMISSIONS AND EXECUTION TIME

Student: Alessandra Vavassori

Student ID Number: 942819

Student mail address: a.vavassori11@campus.unimib.it

Link to Codeberg Repository: https://codeberg.org/Alessandra_1310/Green_Computing_FinalProject.git

Table of contents

1. Introduction
2. Datasets and preprocessing
3. Application of the Random Forest Model
4. Strategies applied to reduce energy, CO₂ emissions and execution time
 - 4.1 Adjustment of the hyperparameters
 - 4.2 Evaluation of an additional strategy: Feature Selection
5. Interpretation of final results
6. Implementation in R programming language
7. Conclusions
8. References

1. Introduction

The aim of this project was to implement effective energy-saving strategies in order to reduce energy consumption, CO₂ emissions and execution time. The project was developed on five datasets which contain data derived from electronic health records. Each dataset included a set of explanatory variables about medical information of the patient and contained also a target variable used to predict whether the patient was affected or not from a specific medical condition.

The method chosen to implement this analysis was the Random Forest algorithm with a repeated hold-out validation of 100 iterations and the accuracy was measured through the Matthews correlation coefficient (MCC). For each dataset it was implemented a “standard code”, with generally a higher number of trees and unlimited maximum depth of the trees, and a “green code” which aimed to reduce energy consumption, CO₂ emissions and execution time by modifying the values of these hyperparameters. Furthermore, an additional strategy based on feature selection ranked by importance was evaluated in the first dataset. However, it was lately discarded due to worse performance with respect to the previous strategy.

2. Datasets and preprocessing

The first dataset aims to predict the presence of neuroblastoma (*outcome*: 1=Yes or 0=No). It is composed of 169 rows and 13 columns with no missing values. The distribution of the target variable shows the presence of 92 patients with no neuroblastoma (*outcome* =0) and 77 patients with neuroblastoma (*outcome* =1); therefore, the classes of the target variable seem overall quite balanced. The dataset contains columns regarding:

- Characteristics of the patients such as *Age* or *Sex*
- Medical information of the patients included with binary variables (*Site*, *Risk*, *Stage*, *Radiation*, *surgical_methods*) and continuous variables (such as *autologous_stem_cell_transplantation* and *time_months*).

Moreover, the variable *site*, which assumed the values 0, 1 or 2 was converted into distinct binary indicators using one-hot-encoding. This prevents the Random Forest from interpreting the values in mathematical order or importance and instead correctly interprets them as purely nominal categories. The second dataset “Belgrade 2021 pediatric brain tumor” aims to predict the state of patients affected from brain tumor. It is composed of 173 rows and 31 columns with no missing values. The target

variable *Status OS* contains 94 rows with the value of 2 and 79 rows with the value of 1; therefore the classes of the target variable appear quite balanced. The dataset contains columns regarding:

- Characteristics of the patients, such as *Age* or *Sex*
- Medical information of the patients including continuous variables (such as *Duration of symptoms before diagnosis (days)*, *Total Dose* or *CSI dose*) and binary variables (*Increased ICP*, *Epileptic seizures* and others). These binary features actually assumed the values 1 and 2 rather than the usual values of 0 and 1. However, since these values did not affect the performance of the model, they were kept with their original values.

Moreover, the variable *Operation type*, which assumed the values 1, 2, 3 or 4 and represents the types of operation, was converted into distinct binary variables using one-hot-encoding, as done in the previous dataset.

The third dataset collects data about patients with colorectal cancer. It is composed of 999 rows and 32 columns with no missing values. The target variable aims to predict the death of the patient and contains 540 values of 0 (*death* = 0) and 391 patients with 1, therefore the classes could be more balanced. The dataset contains columns regarding:

- Characteristics of the patients such as *Age* and *Gender*
- Health status and tumor characteristics including binary variables (*signet_ring*, *lymphovascularinvasion*, *perineural*, *ct*, *rt* and many others) and continuous variables (such as *bian_ma* and *interval_r*).

Moreover, we removed from the dataset the variable *cea* as it was already present its logarithmic value with the variable *log_cea*. Additionally, by doing a preliminary exploratory analysis of the features, the variables *cell_diff*, *mucin_type*, *signet_ring*, *lymphovascularinvasion* and *preneural* contained binary normal values like 0 and 1 but also an unusual value of 9. As the number of rows with the values of 9 were just a few, the rows containing these values were dropped from the dataset, passing from 999 rows to 931 rows.

The fourth dataset collects data about depression heart failure and it is composed of 425 rows and 15 columns with no missing values. The target variable aims to predict the death of the patient and contains 326 values with *death*= 0 and 99 patients with *death*=1; therefore, the classes seem quite unbalanced. The dataset contains information regarding:

- Characteristics of the patients, such as *Age(years)* and *Sex*
- Medical information about the patients including continuous variables (such as *Serum sodium*, *Blood urea nitrogen* and others) and binary variables (*Elevated level of BNP/NT-BNP*, *Hospitalized* (1=Yes, 0=No) and others).

By keeping the dataset unchanged and running the model, the mean MCC obtained assumed an extremely high value (0.994) which was a possible signal of data leakage. Therefore, it was computed the correlation coefficients between the target and the other variables to detect which possible features were causing data leakage. The variables excluded from the model were “*Time from HF to Death (days)*” and “*Time from HF to hospitalization (days)*” which showed the highest values of the correlation coefficient, respectively of -0.858 and -0.31. After removing these variables, the model displayed a more realistic value of mean MCC.

The fifth dataset collects data about sepsis and it is composed of 1257 rows and 16 columns with no missing values. The target variable represents mortality and contains 1128 values of class 0 and 129 values of class 1, indicating that the classes are very unbalanced. The dataset contains information regarding:

- Characteristics of the patients, such as *Age* and *Sex*
- Medical information about the patients including continuous variables (such as *SOFA*, *CRP*, *WBCC* and others), the categorical variable *diagnosis_0EC_IM_2_AC* and the binary variable “*group*”.

3. Application of the Random Forest Model

In order to predict the target variable in all the datasets, it was applied the Random Forest algorithm which is a popular machine learning model especially useful for classification tasks (in this case predicting the death of the patients).

The Random Forest algorithm is based on the creation of multiple decision trees and therefore it is an ensemble learning algorithm. For each decision tree, the algorithm selects a random part of the data using a bootstrap sample, ensuring that each tree is trained on a different version of the training set. Moreover, at each decision node, the algorithm selects a subset of features randomly, allowing to avoid overfitting and to make more accurate predictions by increasing the diversity of trees. Each tree makes its own prediction based on the data on which it was trained on and the final output is obtained by selecting the class that receives the majority of votes across all the trees (in case of classification). The Random Forest algorithm contains different hyperparameters that must be defined before training:

- Number of trees (*n_estimators*): represents the number of decision trees
- Maximum depth (*max_depth*): controls the maximum depth of each tree
- Minimum number of samples (*min_samples_leaf*): defines the minimum number of samples required to create a leaf node (which is the final node of a decision tree)

The choice of using a Random Forest algorithm instead of other models was mainly based on its advantages and on the results obtained after its implementation on the datasets.

In particular, Random Forests can handle different types of data, including both numerical and categorical variables. This was particularly useful as all the datasets contained different medical features, some of them binary while others categorical and numerical.

Furthermore, by adjusting the values of its hyperparameters, it is possible to control the computational cost of the algorithm. For example, while in the “standard” version of the code it was applied a larger number of trees (*n_estimators*) and a higher depth (*max_depth*), in the “green” version of the code both of these values were reduced to obtain lower execution time, CO₂ emissions and energy consumption without significantly impacting the performance metric (MCC).

4. Strategies applied to reduce energy, CO₂ emissions and execution time

4.1 Adjustment of the hyperparameters

In order to reduce execution time, energy consumption and level of emissions, the main strategy adopted consisted in adjusting the hyperparameters of the model.

For each dataset, a specific set of parameters was applied to compare the “standard” version of the code with the “green” version of the same code. The goal was to reduce computational costs while maintaining a similar value of performance, measured through the Matthews Correlation Coefficient (MCC).

The hyperparameters included for the “standard” code were:

- *n_estimators* = 100
- *max_depth* = None
- *class_weight* = 'balanced'

while for the “green” version of the code were:

- *n_estimators* = 20
- *max_depth* = 5
- *min_samples_leaf* = 3
- *class_weight* = 'balanced'

The changes applied to these hyperparameters significantly allowed to reduce the complexity of the trees and the computational effort. In particular, the reduction of the number of trees (*n_estimators*) from 100 to 20 indicates that the algorithm must create 80 fewer trees, significantly reducing the execution time and energy consumed.

Moreover, when the depth of the trees (*max_depth*) is set equal to None in the “standard” code it means that the trees can grow without limits. Therefore, setting the maximum depth equal to 5 in the “green” code allows to create smaller trees, significantly reducing time and energy.

The additional parameter *min_samples_leaf* prevents the model from creating nodes with very few samples which could lead to overfitting. Setting this parameter between 1 and 5 allows for a good generalization and reduction of overfitting.

Finally, the hyperparameter *class_weight*=‘*balanced*’ was introduced in all the datasets since it is particularly useful with imbalanced datasets. This parameter assigns a higher weight to the minority class and a higher one to the majority class so the model is forced to give more attention to the minority class during training.

4.2 Evaluation of an additional strategy: Feature Selection

Another strategy considered to reduce energy consumption, execution time and CO₂ emissions in the “green” code was feature selection based on the ranking of the most important features and retaining only the first ten.

However, after testing this approach on the first dataset, it was observed that, even though energy, CO₂ emissions and execution time were lower than the ones of the standard model, these changes were actually smaller compared to the ones obtained by applying only the strategy of the adjustment of hyperparameters. This was likely due to the additional computation required to select the most relevant important features, consequently increasing the values of energy and execution time.

For this reason, feature selection was not included in the final “green” code and it was not developed for the remaining datasets which instead included only the adjustment of hyperparameters. This last strategy was considered the most effective one to provide the largest reduction of all the values while maintaining a good performance.

5. Interpretation of final results

The following table displayed data related to Mean MCC, Energy Consumption (in Wh), CO₂ Emissions (CO₂eq g) and Execution time (in seconds) for all five datasets. It is important to highlight that these values slightly change every time the code is run, especially for energy, emissions and execution time while the mean MCC remains overall the same. However, the values presented below were collected from the final version of the code and, in general, vary only slightly for every run.

Dataset		Mean MCC	Energy consumption(Wh)	CO ₂ emissions (CO ₂ eq grams)	Execution time (seconds)
Dataset 1 (neuroblastoma)	Standard	0.444	0.21	0.058	42.922
	Green	0.514	0.016	0.004	4.253
Dataset 2 (pediatric brain tumor)	Standard	0.778	0.069	0.019	18.752
	Green	0.799	0.016	0.004	4.175
Dataset 3 (colorectal cancer)	Standard	0.221	0.15	0.042	30
	Green	0.252	0.018	0.005	4.767
Dataset 4 (depression heart failure)	Standard	0.253	0.078	0.022	20.358
	Green	0.334	0.035	0.010	7.16
Dataset 5 (sepsis)	Standard	0.495	0.1118	0.032	29.881
	Green	0.581	0.031	0.0087	7.227

Table 1. Comparison of metrics between “standard” and “green” version of the code.

The data obtained from the CodeCarbon library illustrates a consistent reduction in all the values of energy consumption (in Wh), CO₂ emissions (CO₂eq grams) and execution time (in seconds) but also a slight increase of mean MCC.

The highest decrease is evident for the execution time, where in all the datasets it drops from around 20-30 seconds to around 4-7 seconds. However, constant reductions are also evident for energy consumption and CO₂ emissions, demonstrating that the strategy adopted had effectively made the code greener.

Furthermore, as opposed to what we could expect, in some cases the mean MCC is slightly higher in the “green” code rather than the “standard” code by about 0.02-0.08. This could be explained by the fact that in the “standard” version of the code, with a high number of trees and unlimited depth, the model could actually overfit the data, meaning that it is learning the data “too well” including random noise instead of learning the general pattern to predict the target variable. On the other hand, in the “green” version of the code with lower trees and lower depth, the model is able to generalize better on new data.

6. Implementation in R programming language

The last step of this project consisted of creating a new version of the code with another programming language, in this case R. The same previous operations were translated in R and applied on all the datasets, taking care of the differences between the two languages and the libraries needed.

First, to implement the Random Forest model in Python it is required the Scikit-learn library while in R it must be installed the randomForest package.

The naming of the hyperparameters differs between the two languages: while in Python the number of decision trees and maximum depth are defined respectively by “n_estimators” and “max_depth”, in R the same assume different names like “ntree” and “maxnodes”.

Another difference involves the computation of the MCC. In Python the metric is obtained through the matthews_corrcoef function available in scikit-learn whereas in R the MCC coefficient must be computed manually through the confusion matrix. Therefore, it was installed the caret library to plot the confusion matrix and then computed through the formula of the MCC. The caret package also allowed the repeated hold-out validation with 100 iterations through the createDataPartition function.

The execution time in seconds was computed using the Sys.time() function, given by the difference between the end time after execution and start time.

Carbon emissions were estimated using the reticulate package of R. Specifically, by following the workflow proposed by Utrecht University and adapting it to the project, it was possible to track carbon emissions in the “standard” and “green” code to compare the results. The workflow consisted of loading the CodeCarbon module using the reticulate package, importing the OfflineEmissionsTracker class and configure a tracker with specific parameters (like country_iso_code = "ITA" and measure_power_secs = 30 which defines the frequency at which compute emissions). Finally, the tracker was started and stopped right after execution to compute emissions.

7. Conclusions

In conclusion, this project was interesting to understand how to effectively reduce consumption of energy, CO₂ emissions and execution time by applying the main strategy of adjusting hyperparameters in the correct way. Another alternative of the feature selection was also evaluated but lately discarded for a slight worse result with respect to the main strategy adopted.

Moreover, the results obtained with the Python and the R languages were actually very similar with slight differences. Across all the five datasets, the “green” version of the code in both Python and R always presented much lower values of execution time and CO₂ emissions compared to the “standard” version while maintaining a similar MCC mean.

8. References

- GeeksforGeeks. Random Forest Algorithm in Machine Learning. Retrieved from:
<https://www.geeksforgeeks.org/machine-learning/random-forest-algorithm-in-machine-learning/>
- Snowflake. What Is Random Forest in Machine Learning?. Retrieved from:
<https://www.snowflake.com/en/fundamentals/random-forest/>
- GeeksforGeeks. Hyperparameters of Random Forest Classifier. Retrieved from:
<https://www.geeksforgeeks.org/machine-learning/hyperparameters-of-random-forest-classifier/>
- GeeksforGeeks. Random Forest Approach in R Programming. Retrieved from:
<https://www.geeksforgeeks.org/r-language/random-forest-approach-in-r-programming/>
- GeeksforGeeks. How to Measure Execution Time of Function in R ?. Retrieved from:
<https://www.geeksforgeeks.org/r-language/how-to-measure-execution-time-of-function-in-r/>
- GeeksforGeeks. Feature Selection using Random Forest. Retrieved from:
<https://www.geeksforgeeks.org/machine-learning/feature-selection-using-random-forest/>
- Utrecht University (2023). GitHub Repository. Retrieved from:
<https://github.com/UtrechtUniversity/code-carbon-r>
- Link to Codeberg Repository.
https://codeberg.org/Alessandra_1310/Green_Computing_FinalProject.git
- Link to Python code.
<https://colab.research.google.com/drive/152IE0D4oueKvQZK3EmqX4nZ4ulH7eb8Y?usp=sharing>